UNIVERSIDADE LaSalle

Estrutura de Dados
Estrutura de Dados I

Elgio Schlemer
elgio.schlemer@unilasalle.edu.br

Listas lineares

- Dispostas linearmente na memória
- Cada posição 'n' pode ser calculada matematicamente
- Vetores
  - uso de índice
  - Exemplo vet[2]

DEMONSTRAR

Cálculo de endereço

- Visão da memória
- int v[5];
  - cada int ocupa 4 bytes
  - todo o vetor ocupa 20 bytes na memória
- Onde está v[2]?
  - v começa em 1000
  - v[2] é 1000+2*4 = 1008

DEMONSTRAR

End	Valor
1000	
1004	
1008	
1012	
1016	
1020	
1024	

Criação de vetores em C

- Sempre é necessário saber quantos
- porque sempre é necessário alocar espaço em memória de forma contínua
  - não pode o vetor ter os elementos [0] a [999] no endereço 1000 e os [1000] a [2000] no endereço 23000. Tem que ser sequencial

Alocação estática

- o tamanho do vetor está definido na sua definição
- Exemplo:

```
double vet[1000];
double vet2[TAM];
```

(desde que TAM seja um define)

  - 8000 bytes de memória
    - um double tem 8 bytes

Alocação dinâmica

- Pode-se perguntar quantos elementos, e só então alocar
- Mas sempre todos de uma única vez, pois é vetor
- É necessário o uso de ponteiros
  - Exemplo:

```
double vet[1000]; // estático
double *v2;
v2 = malloc(1000 * sizeof(double));
if (v2==NULL) // erro
```

DEMONSTRAÇÃO

Problemas das listas lineares

- Precisa alocar tudo de uma vez
- Não tem a opção de "criar mais um"
  - O uso da função realloc não é recomendada
- Faça um programa para cadastrar 200 mil livros
  - OK. Cria-se um vetor de 200 mil elementos
- Faça um programa que pergunte quantos livros quer cadastrar
  - OK. Ler a quantidade de livros em uma variável e usar malloc para alocar o vetor no tamanho solicitado

Problemas das listas lineares

- Faça um programa para cadastrar livros. O usuário irá sinalizar quando é o último. Mas é no máximo 200 mil.
  - OK. Cria-se um vetor de 200 mil elementos. Se o usuário cadastrar somente 10 mil, usa-se apenas os v[0] a v[9999]
- Faça um programa para cadastrar livros. Não se sabe quantos. O usuário irá sinalizar quando é o último.
  - NÃO. Aí não pode ser vetor, pois não se tem como alocar todos de uma única vez.
  - outra estrutura deve ser usada para este caso.

Ordenamento de listas

- Para otimizar buscas, pode-se manter um vetor ordenado
- Exemplo:
  - valor 3 existe?

34	5	9	123	-5	4	0	234

  - Não existe. Percorreu todo o vetor para descobrir
  - Agora ordenado. Valor 3 existe?

-5	0	4	5	9	34	123	234

  - Não existe. E chegou a esta conclusão após consultar v[2] que é 4. O 3 deveria estar antes dele

Algoritmos de Ordenamento

- Ordenação interna
  - memória
- Ordenação externa
  - arquivos
- Duas partes
  - comparação
  - troca

Algoritmo da bolha (bubble sort)

- Dois laços
- Sucessivas trocas de [i+1] com [i] se [i] for menor
  - leve otimização com flag "trocou"
- algoritmo fácil de entender

34	5	9	3
5	34	9	3
5	9	34	3
5	9	3	34

5	9	3	34
5	9	3	34
5	3	9	34
5	3	9	34

5	3	9	34
3	5	9	34
3	5	9	34
3	5	9	34

Select sort (seleção)

- talvez o mais intuitivo
- busca-se o menor elemento do vetor
- troca ele com a primeira posição [0]
- repete para subvetor que começa em [1]

```
for (i=0;i<n;i++){
  menor = i;
  for (j = i+1;j<n;j++){
    if (v[j]<v[menor])
       menor=j;
  }
  if (menor != i){
    temp = v[i];
    v[i] = v[menor];
    v[menor] = temp;
  }
}
```

Insert sort (inserção)

```
for (i = 1; i < n; i++) {
        temp = v[i];
        for (j = i - 1; j >= 0; j--) {
                if (v[j] <= temp)
                        break;
                v[j + 1] = v[j];
        }
        v[j + 1] = temp;
}
```

- A partir do segundo elemento
  - insere ele na ordem
  - empurra todos os demais

Matrizes

- Possuem linha e coluna
- Acesso a um elemento especificando qual linha e qual coluna
  - graficamente, uma matriz 3x4

	coluna 0	coluna 1	coluna 2	coluna 3
linha 0				
linha 1				
linha 2				

Como representar matrizes

- Matrizes esparsas
  - economiza memória
- Espaço contínuo na memória
  - como no caso dos vetores
- Usando ponteiros
  - matrizes dinâmicas
  - cada linha pode ter quantidades diferentes de colunas, se quiser

Matrizes esparsas

- só armazena colunas válidas
  - isto é, não armazena nulos
- pode ser implementada sempre como uma matriz N linhas por 3 colunas

Primeira linha:

- qtd de linhas
- qtd colunas
- qtd valores

linhas seguintes

- linha
- coluna
- valor

Exemplo matriz esparsa

coluna 0 coluna 1 coluna 2 coluna 3

linha 0

linha

linha

```
int matriz[10][3];
0][ ]=3;
1]=4;
[2]=5;

mat 20;
mat 75;
mat 45;
mat 975;
mat -3;
```

Como o tamanho em memória dessa matriz é 10 linhas, a primeira linha tem as dimensões e as linhas seguintes os valores. Espaço para representar 9 valores na matriz. Valores em quaisquer posições. Se a matriz que se representa, 3x4, tiver todas as suas posições preenchidas (12 valores) não tem espaço para representar

Matriz esparsa

- útil quando se tem uma matriz potencialmente muito grande, mas com poucos elementos válidos. Exemplo:
  - matriz de 200.000 linhas por 15.000 colunas de valores inteiros
    - Um total de 3.000.000.000 (3 bilhões) de celulas
    - Precisa de 11GiB de espaço em memória para armazenar
  - Mas se apenas 0,2% dela tem valores válidos:
    - 6.000.000 de dados
    - `int mat[6000001][3]` é suficiente para representar
      - Apenas 68MiB de memória
- Problema: o acesso aos valores não é eficiente

Matrizes estáticas

- Espaço contínuo na memória
  - como no caso dos vetores

    ```
    int matriz[2][3];
    ```
- Toda ela está alocada continuamente em memória
- é realizado um cálculo de endereço para achar o valor, como no caso dos vetores.

Matriz Cálculo de endereço

- int m[2][3];
  - cada int ocupa 4 bytes
  - ocupa 24 bytes na memória
- Onde está m[1][1]?
  - m começa em 1000

```
ini+((l*qtdCol)+c)*sizeof(tipo)
```

  - m[1][2] é 1000+((1*3)+2)*4
    - inteiro tem 4 bytes
  - m[1][2] está em 1020

End	Valor	
1000		m[0][0]
1004		m[0][1]
1008		m[0][2]
1012		m[1][0]
1016		m[1][1]
1020		m[1][2]
1024		m[1][3]

Vetores dinâmicos

- O tamanho é alocado posteriormente
- uso do malloc
- Depois de alocar, tudo igual
  - vet[30] pode ser dinâmico ou estático
- Diferença entre estático e dinâmico:
  - apenas a forma e quando foi feita a reserva de memória

Função com Vetores

- A passagem de parâmetros de um vetor em C pode ser desta forma:

```
int zeravet(int v[], int t)
int zeravet(int *v, int t)
```

  - Necessário passar o tamanho do vetor
- Esta forma:

```
int zeravet(int v[40], int t)
```

  - é sem efeito algum

Função com Matrizes estáticas

- Já para matriz é necessário sempre passar a quantidade de colunas de cada linha
  - porque é necessária esta informação para calcular o endereço

```
int zeramat(int m[][40], int l, int c)
```

  - Necessário passar quantidade de colunas dentro dos colchetes e também como parâmetros

Matrizes dinâmicas

- Não está continua na memória
  - apenas colunas estão contínuas
  - na prática é um vetor de vetores
- Quantidade de linhas é dinâmica
- Quantidade de colunas por linha é dinâmica

```
int **mat;
```

DEMONSTRAÇÃO

Definição do Exercício EX1

- Vale 1,0 na G1
- Em grupos
- Envolve vetores de estruturas
- Entregar programa fonte
  - sugestão de link

UNIVERSIDADE LaSalle

Até a próxima aula, dia 11 de abril